

Multi-module programming in ASSEMBLY

- 1) How to call a procedure: `call label` → returns the value of the next instruction
→ jump + puts on stack the return addr.
- 2) How to return from a procedure: `ret` → of the next instr.
↓
transfers program control to a return address located on the top of the stack
- 3) Local label: `.label` (used in a procedure)
- 4) Non-local label: `label:`
- 5) How to include other source files: `%include "factorial.asm"`
- 6) Procedure → defined in the same file as the main → a specific label.
→ ≠ file

lab8_procedure.asm

segment data...

`fmt_string db "factorial=%d", 10, 13, 0`

segment code...

`factorial:`

`mov ecx, 1 ; factorial starts at 1`
`mov ecx, [esp+4] ; esp → return addr.`

• repeat: ; [esp+4] value 6

`mul ecx ; multiply`

`loop repeat ; decrease ecx`
`jump to .repeat`

`ret` ; ret. addr. from stack

Start: ; jump to the instr. after

`push dword 6`

`call factorial` ←

`push eax`

`push fmt_string`

`call [printf]`

`add esp, 4*2`

`push 0`

`call [exit]`

lab8_proc_main.asm

`%include "factorial.asm"`

segment data use32...

`fmt_string db "factorial=%d", 10, 13, 0`

segment code...

Start:

`push dword 6 ; we want 6!`

`call factorial` ; push return address
 jumps to label factorial

`push eax`

`push fmt_string`

`call [printf]`

`add esp, 4*2`

`push 0`

`call [exit]`

factorial.asm (C module)

`%ifndef FACTORIAL_ASM`

`%define FACTORIAL_ASM`

`factorial:`

`mov ecx, 1`

`mov ecx, [esp+4]`

`repeat:`

`mul ecx`

`loop repeat`

`ret 4`

`%endif`

Test
 For next Lab 14: write 3 modules,
 one in C → assemble in console etc.
 * `func` void → `freecore` are input, output

- Requirements:
- 1) segments declared using public qualifier
↳ the code segment (final) is built by concatenation of code segments from each module.
 - 2) data, code labels → global directive (so they can be accessed by other modules)
 - 3) data, code labels → import to the other module using extern directive (inside a module)
 - 4) variables completely defined inside a single module.
! The jump from one module to another: jump / call / ret.
 - 5) program entry point ← "main program".

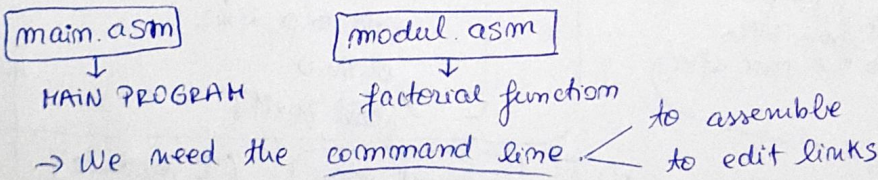
- Each module is assembled separately using: masm - fobj module_name.asm (1)
- How to link modules: ~~alink~~ alink module1.obj ... module n.obj

(2) alink module-1.obj module-2.obj ... -oPE -subsys console -entry start
 ↓ windows console subsystem
 output Win32 PE file (.EXE)
 first instruction to be executed

Stages:

1. ASSEMBLING: (1)
2. LINKING (2)
3. DEBUGGING (3) OLLYDBG.EXE module.exe (3)
4. RUNNING (4) module.exe (4)

Example: Program to compute the factorial of a ~~sum~~ number.



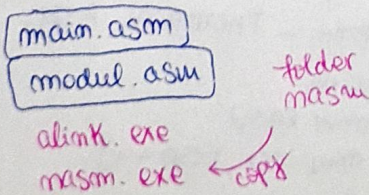
DIAMOND

2 references from that file

↓

define helps with this (compresses)

Disk D → folder asc.



result: d:\asc>main
factorial = 720

→ File → Open → OllyDbg

show the content of the current folder.

Total Commander

d:\asc> cmd

Console: > d: => d:\asc > dir
 > cd asc

⇒ d:\asc> masm -fobj modul.asm (1) → (2) main.obj check visually for ↗
d:\asc> masm -fobj main.asm (2) → (1) modul.obj 1) alink.exe
d:\asc> alink main.obj modul.obj -oPE -subsys console -entry start 2) masm.exe (2)